ML SYSTEM DESIGN

The 9-Step Formula for Production-Grade ML Systems

A Complete Study Guide with Examples, Tips & Interview Strategy

---

What Is ML System Design?

Machine learning system design is the process of designing a complete production-level ML-powered product—not just training a model with good accuracy, but architecting every component needed to build, serve, monitor, and improve it in the real world.

> **The Big Distinction:**
> Research ML: Train a model. Get good metrics. Publish.
> Production ML: Train a model. Serve it to millions. Monitor it. Retrain it. Handle failures. Explain it. Scale it.
> ML System Design interviews test the SECOND kind. That is what this guide is about.

Deploying a deep learning model in production requires several distinct components working together:

- Data pipeline — collection, labeling, storage, and preprocessing
- Feature engineering — converting raw data into model-ready numbers
- Model development — selecting, training, and evaluating the right model
- Prediction service — serving predictions at low latency and high throughput
- Online testing — A/B testing, canary releases, shadow deployment
- Monitoring — tracking data drift, model degradation, system health
- Continuous retraining — keeping the model fresh as the world changes

---

How to Approach an ML System Design Interview

ML system design interviews are open-ended — there is no single correct answer. The interviewer is evaluating your ability to:

1. Translate a vague business problem into a concrete ML problem
2. Design a system that works at scale, not just in a notebook
3. Make and justify trade-off decisions under constraints

4. Communicate clearly — think out loud, ask clarifying questions
5. Know when NOT to use ML (sometimes rule-based is better)

> **Interview strategy tip:**
> Always start by clarifying requirements before jumping into solutions. Interviewers reward engineers who ask the right questions. 'What is the scale? What is the latency requirement? What does success look like?' These questions show maturity.

The 9-Step ML System Design Formula

This is your playbook for every ML system design problem — in interviews and at work. Follow these steps in order, but be flexible: skip steps if the interviewer is not interested, or go deep on one or two if they ask for it.

Step	Name	What You Are Deciding
1	Problem Formulation	What is the ML task? What are the inputs and outputs?
2	Metrics (Offline + Online)	How do we know if the system is working?
3	Architectural Components	What are the main building blocks of the system?
4	Data Collection & Preparation	Where does the data come from? How is it labeled?
5	Feature Engineering	How do we convert raw data into numbers for the model?
6	Model Development & Offline Evaluation	Which model? How do we train and evaluate it?
7	Prediction Service	How do we serve predictions fast and reliably?
8	Online Testing & Deployment	How do we safely roll out the model to users?
9	Scaling, Monitoring & Updates	How do we keep the system healthy and improving?

---

1 Problem Formulation

Translate a business problem into a precise ML task

This is the most important step. Get this wrong, and everything else is built on a broken foundation.

1A — Clarifying Questions to Ask

A **senior** engineer always asks before designing. These questions show you understand what matters in production:

- What is the business goal? (engagement, revenue, safety, efficiency?)
- Who are the users? How many? What latency do they expect?
- What data is available? Volume? Quality? Freshness?
- Are there constraints? (budget, legal, privacy, latency?)
- What does failure look like? (What is the worst outcome?)
- Is real-time prediction required, or can we batch process?

1B — Translate to an ML Problem

After clarifying, you define the ML task precisely:

Component	What to Define	Example
ML Objective	What the model should optimize for	Maximize probability that a user clicks a recommended video
ML Input	What the model receives	User profile + video metadata + context
ML Output	What the model predicts	Probability of click (0 to 1 float)
ML Category	The type of ML task	Binary classification

1C — ML Categories Reference

Category	When to Use	Output Type	Example Problems
Binary Classification	Yes/No decision	Single probability	Spam detection, fraud detection, harmful content
Multi-class Classification	One of N categories	Probability vector (N classes)	Image recognition, language detection
Multi-label Classification	Multiple categories at once	N independent probabilities	Content tagging, disease diagnosis
Multi-task Learning	Multiple related tasks, shared features	N outputs from shared model	Harmful content (violence + nudity + hate)

Category	When to Use	Output Type	Example Problems
Regression	Predict a continuous number	Float value	ETA prediction, house price, ad bid
Ranking / LTR	Order items by relevance	Ranked list	Search results, news feed, recommendations
Embedding / Retrieval	Find similar items efficiently	Dense vector	Semantic search, candidate generation
Sequence-to-Sequence	Transform one sequence to another	Output sequence	Machine translation, summarization

Do we even need ML?
Ask this question out loud in the interview. If the problem can be solved with simple rules (e.g., 'block all posts containing a known list of slurs'), do that first. ML is expensive. Only use it when the problem is too complex for rules or when you have labeled data and scale justifies the investment.

---

2 Metrics (Offline and Online)

Define how you know the system is succeeding

Metrics are the language of alignment between ML engineers and product managers. Define them clearly before building anything.

2A — Offline Metrics (Measured During Development)

These are computed on a held-out test set before the model goes to production.

Classification Metrics

Metric	Formula	When to Use
Accuracy	Correct predictions/total	Only when classes are balanced (rare in practice)
Precision	TP / (TP + FP)	When false positives are costly (spam filter: don't delete good emails)
Recall	TP / (TP + FN)	When false negatives are costly (cancer detection: don't miss cases)
F1 Score	2 × Precision × Recall / (P + R)	When you need a single number balancing precision and recall
ROC-AUC	Area under TPR vs FPR curve	Imbalanced data — overall discrimination ability

Metric	Formula	When to Use
PR-AUC	Area under Precision vs Recall curve	Highly imbalanced data (rare positives) — more informative than ROC
Log-Loss	$-(y \log p + (1-y) \log(1-p))$	When confidence calibration matters, not just ranking

Ranking & Retrieval Metrics

Metric	What It Measures	Best For
Precision@K	Relevant items in top K results	When top results matter most (search)
Recall@K	Relevant items found in top K	When coverage matters (candidate generation)
mAP	Mean Average Precision across queries	Binary relevance, e.g. document retrieval
MRR	Mean Reciprocal Rank of first hit	When only the first correct result matters
nDCG	Normalized Discounted Cumulative Gain	Graded relevance — better results higher up get more credit

Other Offline Metrics

- Regression: MSE (mean squared error), MAE (mean absolute error), RMSE
- NLP tasks: BLEU (translation quality), ROUGE (summarization), GLUE (general NLP benchmark)
- Computational cost: inference latency (p50, p95, p99), memory footprint, FLOPs

2B — Online Metrics (Measured in Production)

These are business-level signals measured on real users. They tell you if the model is actually helping.

Metric	What It Captures	Common Use Cases
Click-Through Rate (CTR)	Fraction of shown items that get clicked	Recommendations, ads, search
Engagement Rate	Likes, comments, shares per impression	Social media feed, content ranking
Conversion Rate	Actions taken (purchase, signup) per session	E-commerce, ads, onboarding
Session Length / Watch Time	How long users engage per session	Video recommendations, music

Metric	What It Captures	Common Use Cases
Task Success Rate	Fraction of sessions where user completed their goal	Search, Q&A, navigation
Revenue Lift	Revenue change vs control group	Ads, pricing, upsell
Counter Metrics (negative)	Hide rate, skip rate, report rate	Content quality, safety

Key Trade-Off:
Optimizing only for CTR can lead to clickbait. Optimizing only for watch time can surface addictive content. Always define counter metrics (negative feedback signals) alongside positive ones. A healthy system maximizes positive engagement while keeping negative signals low.

---

3 Architectural Components (MVP Logic)

Design the high-level system and its main building blocks

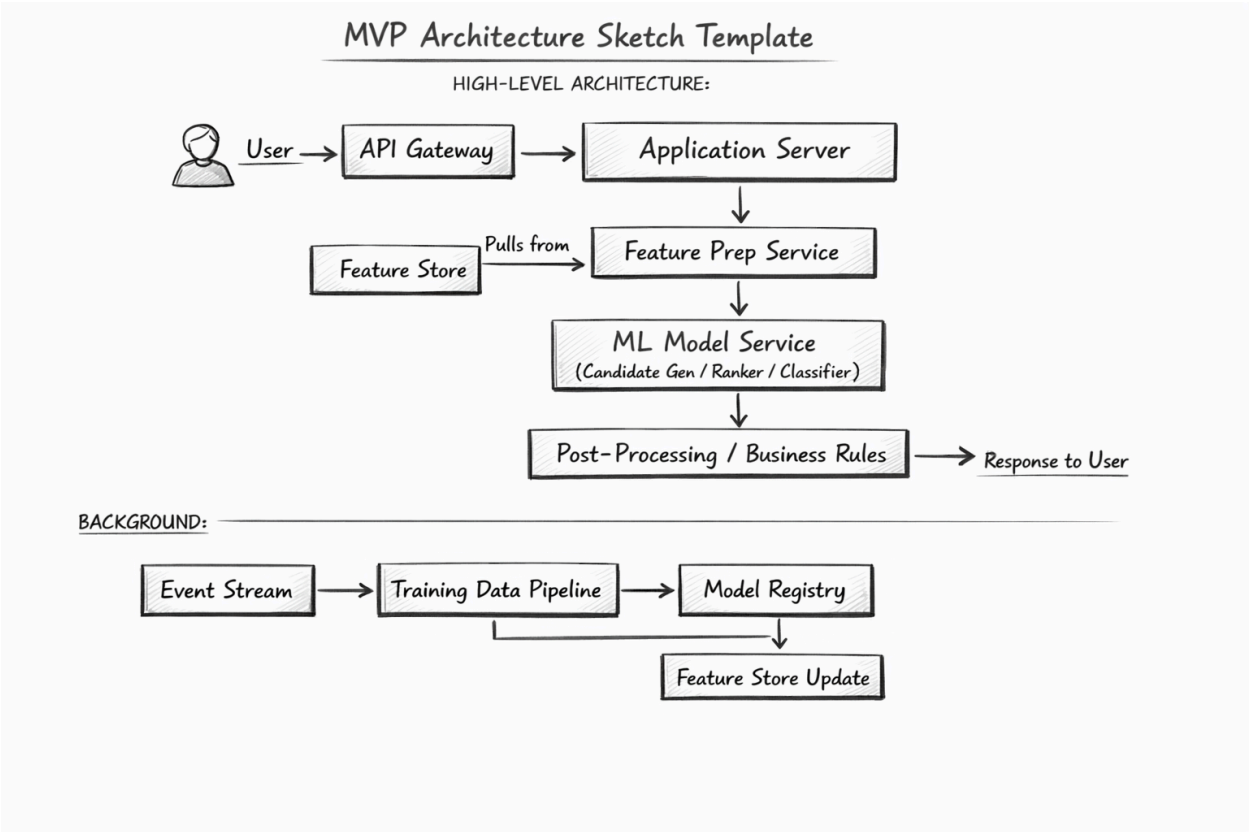
This is the whiteboard moment. Sketch the full system — every component and how they connect. Think in two layers:

- Non-ML components: API servers, databases, message queues, CDNs, user-facing interfaces
- ML components: candidate generators, rankers, classifiers, embedding stores, feature pipelines

3A — Common ML System Architectures

System Type	Typical Architecture Pattern	Examples
Recommendation	Candidate Generation → Ranking → Re-Ranking → Filter	YouTube, Netflix, Spotify
Search	Query Processing → Retrieval (ANN) → Ranking → Result Rendering	Google, LinkedIn Search
Content Moderation	Feature Extraction → Multi-Task Classifier → Enforcement Tiers	Facebook, YouTube Safety
Ads Serving	Targeting → Candidate Retrieval → Bid Prediction → Auction → Ranking	Google Ads, Meta Ads
Newsfeed Ranking	Candidate Pool → Engagement Predictor → Diversity Filter → Feed	Facebook, Twitter/X

3B — MVP Architecture Sketch Template



Interview Tip:
Always draw this diagram on the whiteboard first. Say 'Here is my MVP architecture. Let me walk through each component.' Then go deeper on whichever component the interviewer shows most interest in.

---

4 Data Collection and Preparation

Where data comes from and how it gets labeled

4A — Data Needs Assessment

Before collecting data, define what you need:

- Target variable: What does the model predict? Is it available in the data?
- Key actors: Who or what are the main entities? (users, items, documents, queries, ads)
- Data types: Structured (tabular) or unstructured (text, image, video, audio)?
- Volume: How much data is needed? How much is available?
- Freshness: How quickly does the data become stale?

4B — Labeling Methods

Method	How It Works	Quality	Cost	Best For
Natural Labels	Use implicit user actions (clicks, likes, purchases, watch time)	Noisy — clicking ≠ loving	Free — already logged	Large-scale training data
Explicit User Feedback	Ratings, thumbs up/down, survey responses	Higher quality	Low cost	When users voluntarily give feedback
Human Annotation	Paid contractors manually label data	Gold standard	Very expensive + slow	Evaluation datasets, safety data
Weak Supervision	Heuristics, regex, keyword lists, other models as labels	Noisy but consistent	Cheap + fast	When hand-labeling at scale is impossible
Semi-supervised	Label small set → propagate to unlabeled data via perturbation/similarity	Moderate	Medium	When labeled data is scarce
Transfer Learning	Pre-train on large dataset, fine-tune on task	Very High	Medium	NLP and vision tasks with limited data

4C — Handling Missing Negative Labels

The Missing Negative Problem:
In recommendation systems, a user not clicking a video does NOT mean they disliked it — they may never have seen it. This is a critical problem called 'missing not at random' (MNAR). Naive negative sampling (treating all unseen items as negative) introduces heavy bias. Solution: random negative sampling, popularity-weighted negative sampling, or explicit user-provided negative signals (e.g., 'not interested' button).

4D — Data Splits

- Standard split: 70% train / 15% validation / 15% test
- Time-correlated data: ALWAYS split by time — never shuffle. Future data leaking into training = catastrophic overfitting.
- Data leakage: Compute scaling parameters (mean, std) ONLY on the training split. Apply those same parameters to validation and test.

4E — Data Augmentation

- Images: random crop, horizontal flip, color jitter, rotation, cutout, mixup
- Text: back-translation (translate to another language and back), synonym replacement, random insertion/deletion
- Tabular: SMOTE (synthetic minority oversampling for class imbalance)

5 Feature Engineering

Converting raw data into numbers the model can use

Feature engineering is where domain knowledge meets mathematics. A good feature set often beats a better model with a poor feature set.

5A — Define Your Key Actors

First, identify the main entities in your system. Then define features for each:

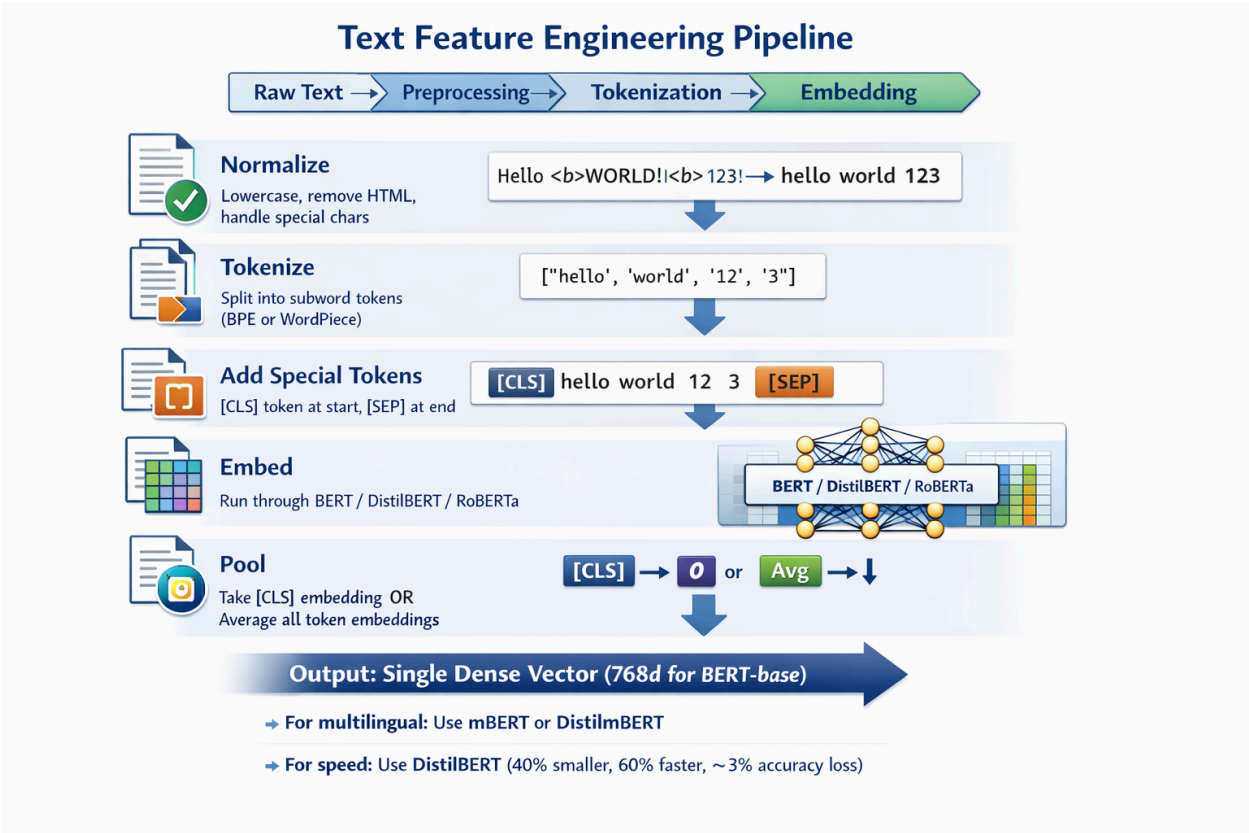
Actor	Example Features	Notes
User	Age, gender, location, account age, recent activity, historical preferences	Profile features + behavioral features
Item / Content	Category, tags, recency, quality signals, embedding	Static metadata + learned representations
Query (Search)	Query text embedding, query length, entity types, intent	Critical for search/retrieval systems
Context	Time of day, device, location, session length, day of week	Captures situational variation
Cross Features (User × Item)	User's past interactions with this item category, user-item similarity	Often the most predictive features

5B — Feature Representation Techniques

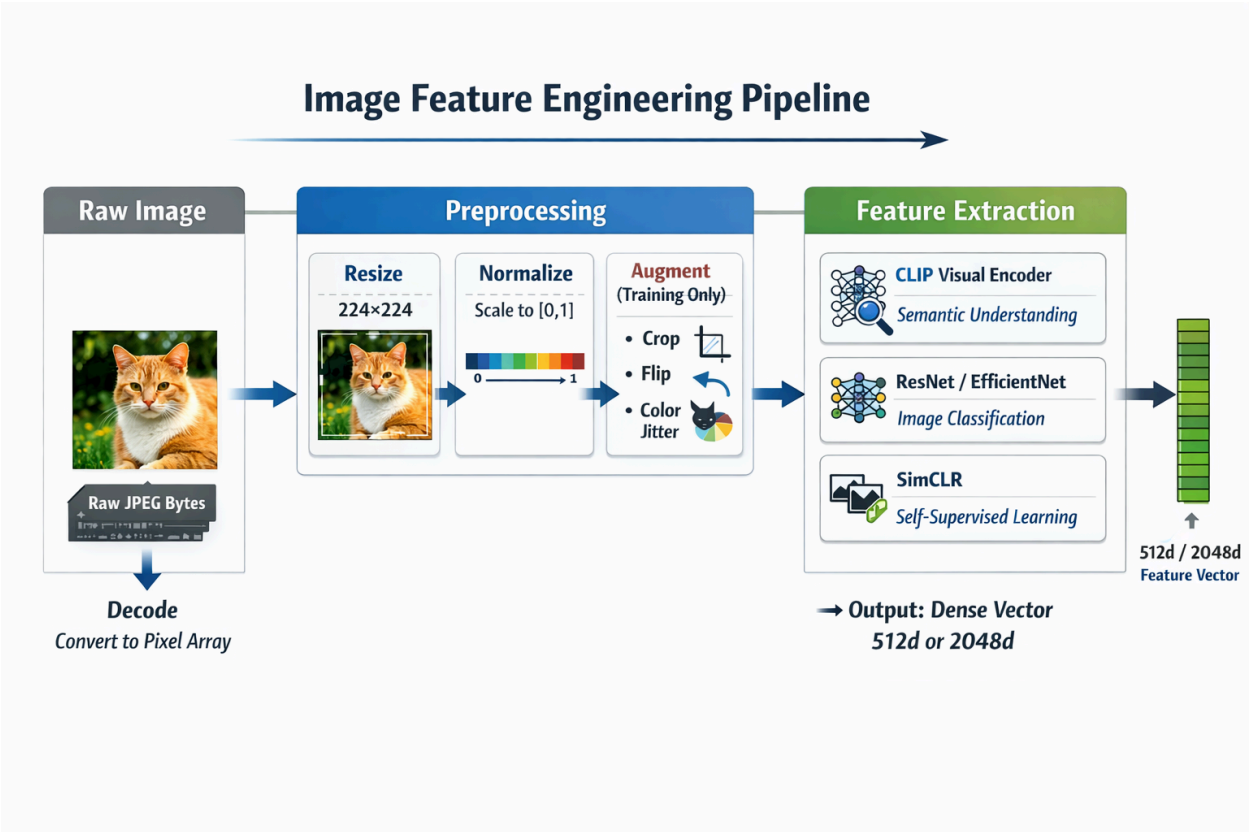
Technique	Input Type	Output	When to Use
One-Hot Encoding	Categorical (low cardinality)	Binary vector	Gender (M/F/Other), device type — few distinct values
Embedding Layer	Categorical (high cardinality)	Dense float vector	User ID, city, video ID — thousands of distinct values
Bucketization + One-Hot	Numerical continuous	Binary vector	Age (0–10, 10–20, ...) to capture non-linearity
Min-Max / Z-score Scaling	Numerical continuous	Scaled float	Any numerical feature — required for neural networks
TF-IDF	Text	Sparse float vector	Keyword importance for document retrieval
Pre-trained Embeddings	Text / Image / Audio	Dense float vector (256–768d)	When semantic meaning matters — BERT, CLIP, Word2Vec

Technique	Input Type	Output	When to Use
Positional Embedding	Sequence data	Position-aware vector	Transformers, any order-sensitive sequence model

5C — Text Feature Engineering Pipeline



5D — Image Feature Engineering Pipeline




```
Raw image → Preprocessing → Feature extraction

1. Decode: convert raw bytes to pixel array
2. Resize: standardize to model input (e.g. 224×224 for ResNet)
3. Normalize: scale pixel values to [0,1] or z-score normalize
4. Augment (training only): random crop, flip, color jitter
5. Extract features via pre-trained model:
   - CLIP visual encoder: great for semantic image understanding
   - ResNet / EfficientNet: strong for classification
   - SimCLR: self-supervised, good for limited labeled data
   → Output: dense vector (512d or 2048d depending on model)
```

5E — Static vs Dynamic Features

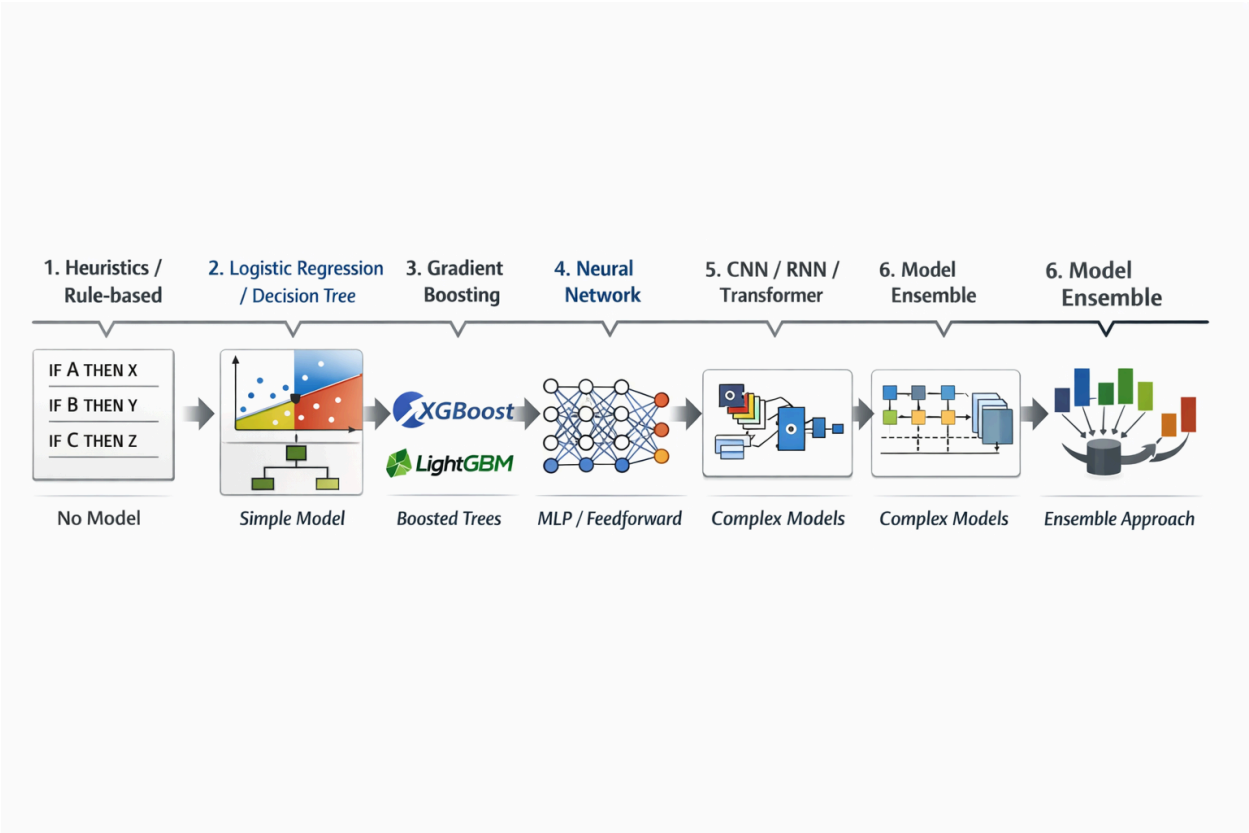
Type	What It Is	Storage	Latency
Static Features	Pre-computed and stored — user profile, item metadata	Feature Store (Redis / DynamoDB)	< 5ms lookup
Dynamic Features	Computed at request time — real-time context, session state	Computed inline in serving	< 10ms compute
Pre-computed Embeddings	Stored model outputs for items (videos, products)	Vector DB / Feature Store	< 5ms lookup

6 Model Development and Offline Evaluation

Choose, train, and evaluate the right model

6A — Model Selection Principle: Always Start Simple

The KISS principle: Keep It Simple and Iterate. Never jump straight to Transformers when logistic regression might work. Build from simple to complex:




```
Iteration path:
  1. Heuristics / Rule-based baseline (no model)
  2. Simple model: Logistic Regression or Decision Tree
  3. Gradient Boosting (XGBoost / LightGBM) for tabular data
  4. Simple Neural Network (feedforward / MLP)
  5. Complex architectures: CNN, RNN, Transformer
  6. Ensemble of models (usually last resort)

Move to the next level ONLY if the simpler model doesn't meet requirements.
```

6B — Model Selection Reference

Model	Best For	Pros	Cons
Logistic Regression	Binary classification, tabular features	Fast, interpretable, no overfitting on small data	Cannot learn non-linear patterns
GBDT / XGBoost	Tabular data (structured features)	Excellent on tabular data, handles missing values, fast	Not ideal for image/text/sequence data
Random Forest	Tabular data, need interpretability	Robust, handles outliers, feature importance built in	Slower than GBDT at inference
Feedforward NN (MLP)	Tabular + combined features	Learns non-linear patterns, flexible	Needs more data than GBDT for tabular
CNN	Image classification, spatial patterns	Captures spatial hierarchy, parameter efficient	Needs GPU, large datasets
RNN / LSTM / GRU	Sequential data, time series	Handles variable-length sequences	Slow training, vanishing gradient issues
Transformer	Text, images, sequences, multimodal	State-of-the-art across almost all domains	Very compute-intensive, needs large data
Two-Tower NN	Retrieval, recommendation, similarity	Efficient ANN search, scalable to billions of items	Requires careful negative sampling

6C — Loss Functions Reference

Loss Function	Task Type	Formula / Notes
Binary Cross-Entropy	Binary classification	$-(y \log p + (1-y) \log(1-p))$ — standard for 0/1 labels
Categorical Cross-Entropy	Multi-class classification	$-\Sigma\, y_i \log p_i$ — use with softmax output
Focal Loss	Imbalanced classification	Down-weights easy examples, focuses on hard ones — great for fraud, content moderation
MSE (Mean Squared Error)	Regression	$\Sigma(y - \hat{y})^2 / n$ — sensitive to outliers

Loss Function	Task Type	Formula / Notes
MAE (Mean Absolute Error)	Regression	$\Sigma\lvert y - \hat{y}\rvert / n$ — robust to outliers
Huber Loss	Regression (robust)	MSE for small errors, MAE for large — best of both
Contrastive Loss	Metric learning, embeddings	Pulls similar pairs together, pushes dissimilar apart
InfoNCE / NT-Xent	Self-supervised contrastive learning	Used in SimCLR, CLIP — excellent for learning embeddings

6D — Handling Class Imbalance

- Oversample minority class: duplicate rare examples in training batches
- Undersample majority class: randomly drop common examples
- Weighted loss function: multiply minority class loss by weight W (e.g. W=10 for 10:1 imbalance)
- Focal loss: automatically focuses training on hard, misclassified examples
- SMOTE: generate synthetic minority examples (works best for tabular data)

6E — Hyperparameter Tuning

- Grid search: exhaustive search over predefined parameter grid — best for small parameter spaces
- Random search: randomly sample parameter combinations — better than grid search for large spaces
- Bayesian optimization: uses past results to guide next trial — most efficient, best for expensive models
- Key hyperparameters: learning rate (most important), batch size, number of layers, hidden dimensions, dropout rate, weight decay

6F — Offline Evaluation Protocol

- Always evaluate on a held-out test set that the model has NEVER seen during training or tuning
- Use stratified splits for imbalanced datasets
- For time-series data: test set must be strictly after training data in time
- Report multiple metrics — never rely on a single number
- Model calibration: predicted probabilities should match actual frequencies (use Platt scaling or isotonic regression if needed)

---

7 Prediction Service

Serving predictions fast, reliably, and at scale

7A — Batch vs Online vs Hybrid Prediction

Mode	How It Works	Latency	Throughput	Best For
Batch Prediction	Pre-compute predictions offline, store results, serve from DB	High (pre-computed)	Very High	Recommendations for known users, offline reports
Online Prediction	Compute prediction at request time	Low (<100ms)	Limited by compute	Real-time personalization, safety, search
Hybrid	Batch for stable features + online for real-time context	Low	High	Netflix: batch video embeddings + online user context

Senior engineer insight:
Most production systems are hybrid. Pre-compute everything that changes slowly (video embeddings, user embeddings). Compute only what must be fresh at request time (current session context, trending items). This gives you low latency AND low cost.

7B — Approximate Nearest Neighbor (ANN) Service

For retrieval systems (search, recommendation candidate generation), you need to find the top-K most similar items from millions or billions of embeddings in milliseconds. Exact search is too slow — use ANN:

ANN Library	Approach	Speed	Accuracy	Best For
FAISS (Meta)	Index-based flat, IVF, HNSW	Very Fast (GPU)	Very High	Large-scale, GPU available
ScaNN (Google)	Anisotropic quantization	Very Fast	High	Production recommendation systems
HNSW (hnswlib)	Hierarchical navigable small world graphs	Fast	High	Managed vector DBs (Pinecone, Weaviate)
Annoy (Spotify)	Random projection forests	Fast	Medium	Read-heavy, no GPU needed

7C — On-Device / Edge AI

When network latency, privacy, or cost prevents cloud inference, models run on the device itself. This requires model compression:

- Quantization: reduce weight precision from float32 to float16 or int8 → 2–4× smaller, 2–4× faster, minimal accuracy loss
- Pruning: remove unimportant weights (near-zero) → sparse model → smaller and faster
- Knowledge distillation: train a small 'student' model to mimic a large 'teacher' model → same behavior, fraction of size

- Factorization: decompose large weight matrices into smaller factor matrices → reduces parameters

7D — Serving Infrastructure

- Model registry: versioned storage of trained models (MLflow, SageMaker Model Registry)
- Model server: serves inference requests (TorchServe, TensorFlow Serving, Triton Inference Server)
- Feature store: centralized store for pre-computed features (Feast, Tecton, Redis + DynamoDB)
- Prediction cache: cache results for repeated queries (Redis, Memcached) — huge cost savings at scale

---

8 Online Testing and Model Deployment

Safely rolling out the model to real users

8A — Deployment Strategies

Strategy	How It Works	Risk Level	Best For
Shadow Deployment	New model runs in parallel, results NOT shown to users — compare predictions silently	Zero	Validating a new model before any user exposure
Canary Release	New model serves a tiny fraction (1–5%) of traffic — monitor metrics closely	Very Low	First real user exposure with easy rollback
Blue-Green Deployment	Two identical environments — switch traffic instantly from old (blue) to new (green)	Low	Zero-downtime deployment with instant rollback
A/B Testing	Split users into control (old model) and treatment (new model) groups, and compare outcomes	Medium	Measuring causal impact of a new model on business metrics
Multi-Armed Bandits	Dynamically allocate more traffic to better-performing variants in real time	Low-Medium	When you want to both test AND optimize simultaneously

8B — A/B Testing Protocol

6. Formulate a null hypothesis: 'The new model has no effect on CTR'
7. Decide the minimum detectable effect: how big must the improvement be to matter?
8. Calculate required sample size using power analysis (typically 80% power, $\alpha=0.05$)
9. Randomly assign users to control and treatment — ensure groups are comparable

10. Run the test until statistical significance is reached (do NOT stop early)
11. Analyze results: primary metrics, counter metrics, segment breakdowns
12. If results are positive: full rollout. If negative: rollback and investigate.

Common A/B testing mistakes:
Peeking problem: checking results before the test is done and stopping early inflates false positive rate
Novelty effect: users behave differently when they see something new — run for long enough to see stable behavior
Network effects: if one user's experience affects another's (social network), standard A/B is biased — use cluster randomization

---

9 Scaling, Monitoring, and Updates

Keeping the system healthy and continuously improving

9A — Scaling the ML System

Scaling Challenge	Solution	Notes
High prediction QPS	Horizontal scaling + load balancer + prediction cache	Cache hit rate of 80%+ dramatically reduces GPU load
Large model, slow inference	Model quantization, batched inference, multi-GPU serving	Batch size tuning critical for GPU utilization
Large training dataset	Distributed training: data parallelism (split data) or model parallelism (split model)	Use NCCL for GPU communication
Feature computation at scale	Pre-compute and store in Feature Store, async feature updates	Avoid recomputing expensive features at request time
Vector search at scale	FAISS/ScaNN with IVF indexing + product quantization	Compress 800M embeddings to fit in RAM with quantization

9B — Monitoring: What to Track

ML systems fail in two ways: software failures and ML-specific failures. Monitor both:

Software System Metrics

- Prediction latency: p50, p95, p99 — alert if p99 exceeds SLA
- Error rate: 4xx and 5xx response rates
- Throughput: requests per second — alert on sudden drops or spikes
- Infrastructure: CPU/GPU utilization, memory, disk I/O

ML-Specific Metrics

- Prediction distribution: monitor the distribution of model outputs — sudden shifts signal problems
- Feature distributions: monitor input feature statistics — data drift here before model degradation
- Online metrics dashboard: CTR, conversion rate, engagement — continuous monitoring
- Model staleness: track how long since last retraining — set maximum allowed staleness

9C — Distribution Shift (The Silent Model Killer)

Shift Type	What It Is	Detection Method	Fix
Covariate Shift	Input feature distribution changes (X changes, P(Y|X) stays same)	Monitor feature statistics (mean, variance, KS test)	Retrain with recent data, importance weighting
Label Shift	Output distribution changes (P(Y) changes)	Monitor prediction distributions and actual labels	Relabel recent data, adjust class weights
Concept Shift	Relationship between X and Y changes (e.g. 'good video' means different thing)	Monitor online metrics degradation	Retrain from scratch with recent labeled data

9D — Continuous Learning / Model Update Strategy

Update Frequency	When to Use	Cost	Freshness
Real-time (online learning)	When user behavior changes within minutes (news, trending topics)	Very High	Maximum — model updates as events arrive
Daily retraining	Most production systems — balance of freshness and cost	Medium	Good — catches daily behavioral shifts
Weekly retraining	When data changes slowly (stable domains)	Low	Acceptable for slow-changing domains
Trigger-based retraining	Retrain when monitoring detects drift above threshold	Low (efficient)	Adaptive — retrain only when needed

9E — Active Learning

When labeled data is scarce and expensive, use active learning to get maximum value from human annotation budget:

- Uncertainty sampling: ask humans to label examples the model is most uncertain about (predicted probability near 0.5)
- Diversity sampling: select examples that are most different from already-labeled examples
- Boundary cases: focus annotation on examples near the decision boundary — highest information gain

Common ML System Design Interview Questions

These are the most frequently asked questions in ML engineering interviews at top tech companies. Use the 9-Step formula on each.

Recommendation Systems

- Video/Movie recommendation (YouTube, Netflix) — Two-Tower + ANN retrieval + ranking
- Friends / follower recommendation (Facebook, Twitter, LinkedIn) — Graph embeddings + PYMK
- Event recommendation (Eventbrite) — Location + time + user interest features
- Game recommendation — Behavioral sequence modeling
- Product replacement recommendation (Instacart) — Category + substitution embeddings

Search Systems (Retrieval + Ranking)

- Text query search — BM25 + semantic search + Learning-to-Rank
- Image / video search — Visual embeddings + multimodal query
- Multimodal search — Joint text-image embedding space (CLIP-style)

Ranking Systems

- Newsfeed ranking (Facebook, Twitter) — Engagement prediction + diversity + freshness
- Ads click prediction — Feature crossing + probability calibration + auction
- Ads serving system — Targeting + retrieval + bid prediction + ranking

NLP Systems

- Autocompletion / typeahead — N-gram + neural language model
- Sentiment analysis — Text classification, BERT fine-tuning
- Named entity recognition — Sequence labeling with CRF or Transformer
- Chatbot / Question answering — Retrieval-augmented generation (RAG)
- Language identification — Lightweight text classifier

Computer Vision Systems

- Image classification — CNN / ViT
- Object detection — YOLO, Faster R-CNN
- Image blurring system (privacy) — Segmentation + face/plate detection

- OCR / text recognition — CNN + CTC decoder

Safety and Trust Systems

- Harmful content detection — Multimodal, multi-task classification (violence, nudity, hate speech)
- Spam detection — Text + behavioral features + graph features
- Fraud detection — Tabular features + graph neural network
- Fake account detection — Behavioral signals + network graph analysis

---

Quick Reference Cheat Sheet

When to Use Which Loss Function

Problem	Loss Function
Binary classification (balanced)	Binary Cross-Entropy
Binary classification (imbalanced)	Focal Loss or Weighted Binary Cross-Entropy
Multi-class (one label per example)	Categorical Cross-Entropy + Softmax
Multi-label (multiple labels per example)	Binary Cross-Entropy per label (sigmoid output)
Multi-task (multiple tasks simultaneously)	Weighted sum of per-task losses
Regression (outlier-sensitive)	MSE
Regression (outlier-robust)	MAE or Huber Loss
Embedding / metric learning	Contrastive Loss or InfoNCE

When to Use Which Evaluation Metric

Situation	Metric to Use
Balanced binary classification	F1, ROC-AUC
Imbalanced binary classification (rare positives)	PR-AUC — more informative than ROC-AUC
Ranking quality (graded relevance)	nDCG
Ranking quality (binary relevance)	mAP, Precision@K
First correct result matters most	MRR (Mean Reciprocal Rank)
Translation / text generation quality	BLEU, ROUGE

Situation	Metric to Use
Calibration (do probabilities match reality?)	Log-loss, Brier score, reliability diagram

Feature Encoding Decision Tree

Feature Type	Cardinality	Encoding
Categorical	Low (< 10 values)	One-Hot Encoding
Categorical	Medium (10–1000 values)	Embedding Layer (learned)
Categorical	High (> 1000 values)	Embedding Layer (learned)
Numerical (continuous)	N/A	Min-Max scaling or Z-score normalization
Numerical (non-linear relationship)	N/A	Bucketization + One-Hot
Text	N/A	Pre-trained BERT / DistilBERT embedding
Image	N/A	Pre-trained CNN / CLIP encoder
Video	N/A	Frame sampling + VideoMoCo / Video Transformer

Reference and Further Reading

Primary Reference:
This document is based on the Machine Learning Interviews repository by Alireza Dirafzoon, which covers ML system design, ML fundamentals, and behavioral interview preparation in comprehensive depth.

GitHub Repository: https://github.com/alirezadir/Machine-Learning-Interviews

Additional Resources

- **Full Stack Deep Learning course:** https://fall2019.fullstackdeeplearning.com/

- **Production Level Deep Learning (GitHub):** https://github.com/alirezadir/Production-Level-Deep-Learning
- **Machine Learning Systems Design (Chip Huyen):** https://github.com/chiphuyen/machine-learning-systems-design
- **Stanford CS329S — ML Systems Design:** https://online.stanford.edu/courses/cs329s-machine-learning-systems-design
- **Grokking the System Design Interview (Educative):** https://www.educative.io/courses/grokking-the-system-design-interview
- **System Design Primer (GitHub):** https://github.com/donnemartin/system-design-primer

Contact & Connect

Email: adilshamim696@gmail.com

LinkedIn: https://www.linkedin.com/in/adilshamim8

GitHub: https://github.com/AdilShamim8

Kaggle: https://www.kaggle.com/adilshamim8

Twitter/X: https://x.com/adil_shamim8

Medium: https://adilshamim8.medium.com/